Microphone, Camera, and backround app

⚠️ Reality Check (VERY IMPORTANT)

❌ You CANNOT:

- Directly access kernel
- Constantly record mic/camera in background silently
- Override system buttons freely (like power button globally)

✅ You CAN:

- Use **Foreground Services**
- Use **Accessibility / Sensors / Permissions**
- Use **OS-approved triggers (partially)**

🧠 1. HOW YOUR APP RUNS "ALWAYS" (Background Execution)

✅ Use: Foreground Service (Android)

This is the most important concept.

How it works:

- App runs as a **persistent service**
- Shows a small notification like:

> "Rakshika is active for your safety"

Why needed:

- Prevents OS from killing your app
- Allows:
  - Sensor monitoring
  - Location updates
  - Trigger detection

Flow:

```
App starts → Foreground Service ON → Runs continuously
```

⚡ 2. TRIGGER DETECTION (REALISTIC IMPLEMENTATION)

🔘 Power Button (Reality)

👉 You **CANNOT directly detect power button presses reliably** (Android restrictions)

Workarounds:

Option 1: Volume Button Pattern (BEST)

- Detect volume up/down presses
- Works reliably

Option 2: Accessibility Service (Advanced)

- Can detect some system interactions
- But:
  - Needs user permission
  - Google Play restrictions

📳 Shake Detection (BEST + EASY)

Use:

- Accelerometer sensor

Logic:

```
If shake intensity > threshold 3 times
→ Trigger emergency
```

🎯 Recommended Final Trigger Strategy

👉 For hackathon:

- Shake detection ✅
- Volume button pattern ✅
- Custom on-screen quick button (backup) ✅

🎤 3. MICROPHONE ACCESS (IMPORTANT LIMITS)

❌ Not Allowed:

- Continuous background recording without user knowledge

✅ Allowed:

After trigger activation:

```
Trigger → App comes to foreground → Start recording
```

Use:

- Android MediaRecorder / AudioRecord

🎯 Best Practice:

- Show small UI or notification:

  > “Recording for emergency”

📷 4. CAMERA ACCESS

Same rule as microphone:

❌ Not allowed:

- Silent background video capture

✅ Allowed:

- Start camera after trigger
- Use:
  - CameraX API (Android)

📍 5. LOCATION TRACKING (CRITICAL FEATURE)

Use:

- FusedLocationProviderClient (Google API)

Modes:

1. Normal Mode:

- Low battery usage

2. Emergency Mode:

```
High accuracy ON
Update every 3–5 sec
```

Permissions Needed:

- ACCESS_FINE_LOCATION
- ACCESS_BACKGROUND_LOCATION

🔁 6. KEEPING APP ALIVE (ANTI-KILL STRATEGY)

Modern Android kills apps aggressively, so:

You must combine:

✅ Foreground Service

✅ Battery Optimization Exclusion

✅ Auto-restart on boot

Boot Receiver:

```
Phone restarts → App auto starts → Service ON
```

🔐 7. PERMISSIONS YOU WILL NEED

Core Permissions:

```
INTERNET
ACCESS_FINE_LOCATION
```

```
ACCESS_BACKGROUND_LOCATION
RECORD_AUDIO
CAMERA
FOREGROUND_SERVICE
RECEIVE_BOOT_COMPLETED
```

Optional (Advanced):

```
SYSTEM_ALERT_WINDOW (for popup)
BIND_ACCESSIBILITY_SERVICE
```

⚙️ 8. REAL EXECUTION FLOW (OS LEVEL SIMPLIFIED)

🔁 App Idle State

- Foreground service running
- Sensors active (low power)

⚡ Trigger Happens

- Shake / button detected
- App wakes up

🚨 Emergency Mode

```
1. Show quick UI
2. Start location tracking
3. Start mic/camera (if needed)
4. Send data to backend
```

📡 Background Communication

- Use:
  - Firebase Cloud Messaging (FCM)
  - REST APIs

🔥 9. WHAT YOU SHOULD SAY IN HACKATHON (VERY IMPORTANT)

👉 Don’t say “kernel level”

Say this instead:

“We use Android foreground services combined with sensor listeners and permission-based APIs to ensure continuous background monitoring while respecting OS security policies.”

⚡ Mentor-Level Insight

If you say “kernel level”, judges will:

- ❌ Reject as unrealistic
- ❌ Ask deep OS questions

If you say:

- Foreground service ✅
- Sensor-based triggers ✅
- Permission-safe architecture ✅

👉 You sound like a **real engineer**

🚀 FINAL RECOMMENDED STACK (FOR YOU)

Since you're Java Full Stack:

Mobile:

- Kotlin (preferred) or Java
- Android SDK

Backend:

- Spring Boot

Realtime:

- Firebase